

SmartCode Browser

级联式代码浏览器

一个人 + AI，做出一个真能用在内核 / 芯片大库上的代码导航工具
几乎没手写代码 —— 全程用 Cursor「聊」出来

为什么需要 · 怎么用 AI 造的 · 要注意什么 · 往哪走

一条线串起来：从缝隙到工具，再到 AI 怎么造的



重点在第 3 段：这工具本身只是个例子，更想分享的是 —— 现在你也能这样用 AI 造出真能用的内部工具。

「调查一条调用链」是被现有工具组织得最烂的任务

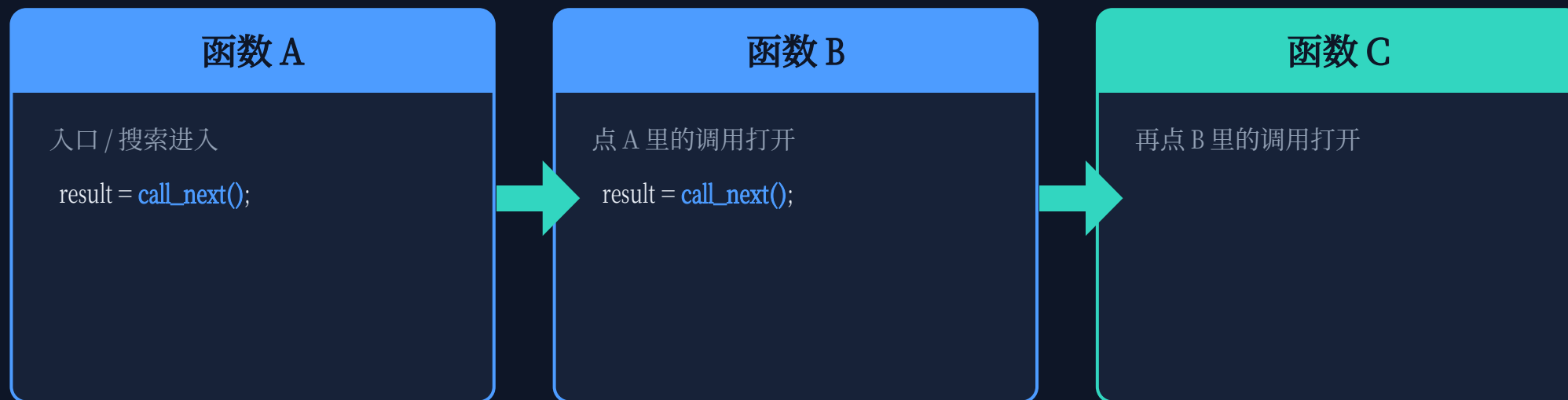
现有方式	能干嘛	追一条链时的别扭
IDE 跳转	准、能改代码	标签越开越多，路径全在脑子里，难复述给别人
grep / 全文搜索	快、无需索引	只给「出现位置」，没有调用层次
直接问 AI	出解释快	行号 / 路径不稳定，难逐行核对

缝隙：不是工具不好，而是「沿调用链调查」这件事，信息组织方式不对。

我们想要的：从入口出发，顺着「谁调谁」一层层展开，路径可见、可续、可分享 —— 像在树上散步。

怎么填这个缝隙

核心体验：点蓝色调用，右侧长出新面板



新面板向右生长，左侧不被盖住 —— 整条调用链 $A \rightarrow B \rightarrow C$ 始终可见、可回看、可截图分享。

它能做什么

能力一览

能力	一句话说明
调用 → 定义	点蓝色引用，级联展开；多实现时真实现优先、空桩靠后
变量 → 声明	局部变量本面板高亮，全局 / 字段 / 枚举开新面板
查找用法	Alt/Ctrl+点击 或右键，看谁在用这个符号
符号搜索	支持 in:路径 限定范围，从任意入口开始追链
多 arch 消歧	配 compile_commands.json 后跳到真正被编译的那一份
会话 / 标签	刷新自动恢复 + 命名保存探索路径，方便交接分享
可选 AI 分析	对当前函数提问，走 cursor-agent（复用 Cursor 登录）

语言无关：C/C++ · Python · Java · Scala/Chisel —— 换语言、换项目主要靠配置。

现场演示

三分钟看明白（重点两个动作）

1

搜入口 → 连点 2~3 层调用

面板一块块向右长出来 —— 全场最直观的瞬间，口播调用链

2

保存标签 → 刷新恢复

命名保存路径 → F5，阅读树自动恢复，强调「可续」



吃自己的狗粮

当场用这个工具去读它自己的源码 —— AI 做的工具来读代码，最有记忆点

其它（查用法、多候选选真实现、AI 提问）视时间补充，别让演示掉节奏。

PART · 重点

怎么用 Cursor 把这个工具「聊」出来

≈ 6900

行代码

8

个 Python 模块

~2400

行前端

7+

次能力迭代

几乎全程 AI 结对完成 —— 人几乎没手写实现，只在「说清楚要什么」和「把关」上花时间。

核心理念：不写实现，写「想要什么」

从真实痛点出发

- 起点是「读 cpuidle 调用链很累」
- 不是「我想学个新框架」
- 需求来自真实工程，验收也回真实工程

用自然语言描述需求

- 「点调用就在右边展开定义」
- 「同名多实现时真实现优先」
- AI 负责翻译成跨文件的代码

人审 + 小步迭代

- 每个能力一个 commit，独立验证
- 跑起来不对就回退、重新描述
- 实现是 AI 的活儿，方向是人的活儿

一句话：把 Cursor 当「随叫随到的结对工程师」——你负责想清楚，它负责写出来。

真实迭代时间线（每条 = 一次对话 + 一个 commit）



怎么造的 (3/4)

具体怎么和 Cursor 配合

Agent / Composer 多文件改

一句需求同时改后端 + 前端，保持端到端一致

@ 引用精确喂上下文

@文件 / @符号 指给 AI，少绕弯、改得准

边写边跑自检

curl 接口 + 浏览器实时看效果，错了立刻反馈给 AI

SpecStory 留存对话

.specstory/ 自动记录每次会话 = 开发决策日志

吃自己的狗粮

registry 默认指向本仓库，用工具读工具自身代码

让 AI 同步写文档

README、这份 PPT 讲稿都让 AI 跟着代码一起更新

人和 AI 各干什么

人负责（少而关键）

- 定方向：解决哪个真实痛点
- 定边界：只读 / 语言无关 / 不做 IDE
- 定优先级：如「真实现优先于空桩」
- 验真场景：拿内核 / NEMU 实测
- 审结果、把关取舍与质量

AI（Cursor）负责（重活）

- 几乎全部实现：跨文件写 / 改代码
- 脚手架：服务、前端、语言适配
- 补盲区：宏、函数指针等边界 case
- 重构、命名、写注释 / 文档
- 解释报错、给修复方案

AI 写了几乎所有代码，但「这东西该是什么样」始终是人在拿主意。

用 AI 做这类东西，要注意什么

① 方向与边界：人来守

- 场景要具体（需求 $\approx$ 验收标准）
- 边界钉死，否则范围一路膨胀
- 优先级是人的判断，AI 给不了

② 质量：别信「看起来对」

- 每次都审 diff，尤其跨文件
- 小步提交 + 拿真实数据验证
- 防幻觉：编造的 API 去查文档

③ 上下文：喂得准、会漂

- @文件 / @符号 精确给上下文
- 长对话会忘约定，适时开新会话
- 把边界 / 风格固化进规则文件

④ 安全与合规

- 别把密钥 / 内部源码无意识喂出去
- 生成代码可能撞开源，关键处留意
- 危险操作（删库 / 强推）人来确认

一句话：AI 是放大器 —— 放大你的判断力，也放大你的疏忽。坦诚讲局限，反而最加分。

这不是玩具

几个为真实工程做的取舍（AI 实现，人来定）

多 arch / 多 ISA 消歧

同名实现 x86 / riscv / mips 各一份；读 compile_commands.json 判断「这次实际编译了哪份」，稳定命中真实现

真实现优先、空桩靠后

大库里同名 / 弱符号 / #ifdef 桩满天飞 —— 候选排序体现对真实工程的理解

诚实优于炫技

静态解析有盲区，解析不了就明确标注、绝不假装能跳；grep 兜底保证至少搜得到名字

后面打算做什么

近期 · Now/Next

难解析场景增强：函数指针 / 表驱动 / 宏 跳得更准

全局调用图视图：不止级联，整张图一眼看全

调用树导出 / 分享链接：把路径发给同事

中期

更深 AI：一键讲解整条调用链

对调用图提问：「谁最终调用了它」

接 IDE / Git / PR：跳编辑器、评审挂调用树

远期

更多语言：Rust / Go / Verilog 完善

团队协作：登录鉴权 / 共享标签

性能：超大库增量索引、更快冷启动

一页带走

缝隙 → 工具

「沿调用链调查」被现有工具组织得很烂；级联交互把它变成沿调用树散步

怎么造的（重点）

一个人 + Cursor 聊出来：人提需求 + 把关，AI 写几乎全部实现，小步迭代

最想分享的

门槛降了 —— 你也能这样造内部工具；但方向、边界、把关，仍是人的活儿

谢谢 · 欢迎提问

现场演示 + Q&A